TP : CSS3 – Transform, Filtres, Transitions et Animations

Classe : 3ème SI

Sommaire

Exercice 1 : Transition : le bouton qui réagit au survol
Exercice 2 : Fonctions de progression : linear, ease, ease-in, ease-out
Exercice 3 : Animation avec @keyframes et forwards
Exercice 4 : Plusieurs transform combinés (2 à 4)
Exercice 5 : Filtres CSS : blur, grayscale, invert
Exercice 6 : Plusieurs animations en même temps
Exercice 7 : Formes en mouvement : cercle, carré, triangle
Exercice 8 : La roue qui avance et reste en rotation
Exercice 9 : Projet final : page d'accueil animée (flex, sticky, fixed)

Consignes

- Pour chaque exercice, créez un fichier HTML et un fichier CSS à partir du code de départ.
- Suivez les étapes dans l'ordre et testez dans le navigateur après chaque étape.
- Répondez aux questions directement sous chaque question.
- Les bonus sont facultatifs.

Rappel de cours

Transition et animation

Transition : passage progressif d'un style à un autre (souvent au survol). `transition: propriété durée fonction délai;`

Animation : on décrit les étapes avec @keyframes (from/to ou 0 % … 100 %), puis on l'applique : `animation: nom durée fonction délai répétition direction fill-mode;`

Plusieurs animations sur un même élément : on les sépare par une virgule.

animation-fill-mode

Valeur	Effet
`none`	Valeur par défaut : à la fin, l'élément revient à son style initial.
`forwards`	À la fin, l'élément garde le style de la dernière image clé.
`backwards`	Pendant le délai, l'élément prend le style de la première image clé.
`both`	Combine forwards et backwards.

Fonctions de progression (timing-function)

Valeur	Comportement
`linear`	Vitesse constante du début à la fin.
`ease`	Valeur par défaut : démarre vite, ralentit longuement vers la fin.
`ease-in`	Démarre lentement, puis accélère jusqu'à la fin.
`ease-out`	Démarre vite, puis ralentit jusqu'à l'arrêt.
`ease-in-out`	Lent au début, rapide au milieu, lent à la fin.

Fonctions de transform

Fonction	Effet
`translate(x, y)`	Déplace l'élément (translateX, translateY pour un seul axe).
`rotate(angle)`	Fait pivoter l'élément (ex. 45deg).
`scale(n)`	Agrandit (n > 1) ou réduit (n < 1).
`skewX(angle)`	Incline l'élément horizontalement.

La propriété filter

Fonction	Effet
`blur(4px)`	Rend l'élément flou ; plus la valeur est grande, plus il est flou (0 = net).
`grayscale(100%)`	Passe en noir et blanc (0 % = couleurs d'origine, 100 % = gris complet).
`invert(100%)`	Inverse les couleurs (blanc ↔ noir, rouge ↔ cyan) ; 0 % = aucun effet.

On peut combiner plusieurs filtres, séparés par des espaces : `filter: grayscale(100%) blur(2px);` ; `filter: none;` les supprime.

La propriété position

Valeur	Effet
`static`	Valeur par défaut : l'élément suit le flux normal.
`relative`	Décalé par rapport à sa place normale ; sert de repère aux éléments absolute.
`absolute`	Sorti du flux ; positionné par rapport au parent positionné le plus proche.
`sticky`	Suit le flux, puis « colle » dans la fenêtre dès qu'il atteint top (ou bottom).
`fixed`	Sorti du flux ; positionné par rapport à la fenêtre, il ne bouge pas au défilement.

Exercice 1 : Transition : le bouton qui réagit au survol

Objectif : *créer un effet progressif au survol avec transition.*

Code de départ

```
<button class="btn">Survolez-moi</button>

.btn {
  background: #2c3e50;
  color: #fff;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

Travail demandé

1. Au survol (:hover), le fond devient `#e74c3c` et le bouton s'agrandit avec `transform: scale(1.1)`.
2. Testez : le changement est-il progressif ou brusque ?
3. Ajoutez une transition de 0.4s avec ease-in-out dans la règle .btn (et non dans :hover). Comparez.
4. Ajoutez un délai (transition-delay) de 0.2s. Que remarquez-vous ?

Questions

Question 1 : Pourquoi place-t-on la transition dans .btn et non dans .btn:hover ?

Question 2 : Quelle est la différence entre transition: all 0.4s et transition: background 0.4s ?

Exercice 2 : Fonctions de progression : linear, ease, ease-in, ease-out

Objectif : *comprendre comment la fonction de progression change la vitesse d'une animation.*

Code de départ

Quatre balles font la course sur quatre lignes.

```
<div class="piste">
  <div class="ligne"><div class="balle linear">linear</div></div>
  <div class="ligne"><div class="balle ease">ease</div></div>
  <div class="ligne"><div class="balle ease-in">ease-in</div></div>
  <div class="ligne"><div class="balle ease-out">ease-out</div></div>
</div>

.piste {
  width: 600px;
  border-right: 4px solid #e74c3c;   /* ligne d'arrivée */
}
.ligne { border-bottom: 2px dashed #bbb; padding: 8px 0; }
.balle {
  width: 70px; height: 34px;
  line-height: 34px; text-align: center;
  border-radius: 17px;
  background: #3498db; color: #fff;
  font-size: 13px;
}
```

Travail demandé

1. Créez `@keyframes course` : from → `transform: translateX(0);` ; to → `transform: translateX(500px);`.
2. Appliquez l'animation aux quatre balles : 4s, forwards, mais une fonction différente par balle (linear, ease, ease-in, ease-out), une classe par balle.
3. Rechargez la page plusieurs fois et observez les quatre balles.

Questions

> **Question 1 :** Après 1 seconde, quelle balle est la plus avancée ? Laquelle est la plus en retard ?

> **Question 2 :** Toutes les balles arrivent-elles en même temps ? Que change la fonction de progression : la durée ou la façon de parcourir la distance ?

Question 3 : Quelle fonction choisiriez-vous pour : (a) un message qui entre dans la page ; (b) un message qui sort de la page ; (c) une barre de chargement qui avance régulièrement ?

Bonus

- Ajoutez une cinquième balle avec ease-in-out.
- Ajoutez une septième balle avec steps(5) (mouvement par paliers).

Exercice 3 : Animation avec @keyframes et forwards

Objectif : *comprendre le rôle de animation-fill-mode: forwards.*

Code de départ

```
<div class="boite">Je me déplace</div>
<div class="message">Bienvenue sur mon site !</div>

<div class="barre">
  <div class="remplissage"></div>
</div>
```

```
.boite {
  background: #3498db; color: #fff;
  padding: 15px; width: 150px;
  margin-bottom: 20px;
}
.message {
  background: #2ecc71; color: #fff;
  padding: 20px; width: 250px;
  margin-bottom: 20px;
}
```

Travail demandé

1. Créez `@keyframes deplacer` : from → `translateX(0); background: #3498db;` ; to → `translateX(200px); background: #e74c3c;` (la propriété transform pour translateX).
2. Appliquez-la à .boite : `animation: deplacer 2s ease;`. Observez la fin de l'animation.
3. Ajoutez `forwards` : `animation: deplacer 2s ease forwards;`. Observez de nouveau.
4. Créez `@keyframes apparition` : from → `opacity: 0; transform: translateY(-50px);` ; to → `opacity: 1; transform: translateY(0);`.
5. Dans .message : `opacity: 0;` puis `animation: apparition 2s ease;` (sans forwards). Observez la fin.
6. Corrigez le problème avec forwards.

7. Barre de progression : .barre (largeur 300px, hauteur 20px, fond gris) ; .remplissage (hauteur 100%, fond vert, largeur 0) ; animation `charger` : width de 0% à 80% en 3s, linear, avec forwards. La barre doit rester remplie à 80 %.

Questions

Question 1 : Que se passe-t-il à la fin de l'animation deplacer sans forwards ? Et avec forwards ?

Question 2 : Expliquez avec vos mots le rôle de forwards.

Question 3 : Pourquoi le message disparaît-il à la fin de l'animation sans forwards ?

Bonus

- Mettez le from de charger à width: 20%, ajoutez animation-delay: 1s, puis testez backwards et both.
- Comparez ce que vous voyez pendant le délai pour none, backwards et both.

Exercice 4 : Plusieurs transform combinés (2 à 4)

Objectif : *combiner de 2 à 4 fonctions dans une même propriété transform et comprendre que l'ordre compte.*

Code de départ

```
<div class="carte">Survolez-moi</div>

<div class="repere">
  <div class="carre ca">A</div>
  <div class="carre cb">B</div>
</div>

<div class="piste-danse">
  <div class="danseur">★</div>
</div>
```

```
.carte {
  width: 160px; padding: 30px 10px;
  margin: 40px;
  text-align: center;
  background: #9b59b6; color: #fff;
  border-radius: 8px;
}
.repere {
  position: relative;
  width: 400px; height: 250px;
  margin: 20px 40px;
  border: 2px dashed #999;
}
.carre {
  position: absolute; top: 20px; left: 20px;
  width: 50px; height: 50px;
  line-height: 50px; text-align: center;
  color: #fff;
}
.ca { background: #e74c3c; }
.cb { background: #2980b9; }
.piste-danse {
  position: relative;
  width: 600px; height: 120px;
  background: #f4f4f4;
}
.danseur {
  position: absolute; top: 35px; left: 10px;
  width: 50px; height: 50px;
  line-height: 50px; text-align: center;
  font-size: 24px;
  background: #f39c12; color: #fff;
}
```

Travail demandé

1. Deux transformations : au survol de .carte, `transform: translateY(-15px) scale(1.1);` avec une transition de 0.4s.
2. Trois transformations : ajoutez une rotation : `translateY(-15px) rotate(5deg) scale(1.1)`. Comparez.
3. L'ordre compte : donnez aux carrés une transition de 1s sur transform. Au survol de .repere : A reçoit `translateX(150px) rotate(45deg)` et B reçoit `rotate(45deg) translateX(150px)`. Survolez le cadre.
4. Quatre transformations : créez `@keyframes danse` avec les valeurs ci-dessous (mêmes fonctions, même ordre à chaque étape), puis appliquez `animation: danse 5s ease-in-out forwards;` à .danseur.

```
0%   : translateX(0)     rotate(0deg)   scale(1)   skewX(0deg)
33%  : translateX(150px) rotate(120deg) scale(1.3) skewX(15deg)
66%  : translateX(300px) rotate(240deg) scale(0.8) skewX(-15deg)
100% : translateX(450px) rotate(360deg) scale(1)   skewX(0deg)
```

Questions

Question 1 : A et B utilisent les mêmes fonctions : pourquoi n'arrivent-ils pas au même endroit ?

Question 2 : Pourquoi doit-on écrire les quatre fonctions, dans le même ordre, à chaque étape de l'animation ?

Bonus

- Ajoutez transform-origin: bottom left; à .carte et observez la différence avec le centre (valeur par défaut).

Exercice 5 : Filtres CSS : blur, grayscale, invert

Objectif : *modifier l'aspect d'un élément avec filter (blur, grayscale, invert) et le faire changer progressivement.*

Code de départ

```
<div class="galerie">
  <div class="photo p1">Flou</div>
  <div class="photo p2">Gris</div>
  <div class="photo p3">Inversé</div>
  <div class="photo p4">Gris + flou</div>
</div>

.galerie { display: flex; gap: 20px; }
.photo {
  width: 160px; height: 120px;
  border-radius: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
  color: #fff;
  background: conic-gradient(#e74c3c, #f1c40f, #2ecc71,
                             #3498db, #e74c3c);
}
```

Travail demandé

1. Dans .photo, ajoutez une transition : `transition: filter 0.5s;`.
2. blur : .p1 a `filter: blur(4px);` et au survol `filter: blur(0);`.
3. grayscale : .p2 a `filter: grayscale(100%);` et au survol `filter: grayscale(0%);`.
4. invert : au survol de .p3, `filter: invert(100%);`.
5. Combinaison : .p4 a `filter: grayscale(100%) blur(2px);` et au survol `filter: none;`.
6. Animation : créez `flou-net` (from : opacity 0 et blur(12px) ; to : opacity 1 et blur(0)) et appliquez-la à .galerie : `animation: flou-net 2s ease forwards;`.

Questions

Question 1 : Que se passe-t-il quand la valeur de blur augmente ? Que donne blur(0) ?

Question 2 : Quelle différence y a-t-il entre grayscale(50%) et grayscale(100%) ?

Question 3 : Que donne invert(100%) sur du blanc ? sur du noir ? sur du rouge ?

Question 4 : Le filtre s'applique-t-il seulement au fond de l'élément ou aussi à son texte ?

Bonus

- Essayez invert(50%) : quelle couleur obtenez-vous ?

Exercice 6 : Plusieurs animations en même temps

Objectif : *appliquer plusieurs animations à un même élément et comprendre le conflit sur une même propriété.*

Syntaxe : on sépare les animations par une **virgule** : `animation: anim1 4s linear forwards, anim2 2s ease infinite;`

Code de départ

```
<div class="ciel">
  <div class="soleil"></div>
</div>
```

```
.ciel {
  position: relative;
  width: 600px; height: 200px;
  background: #87ceeb;
  overflow: hidden;
}
.soleil {
  position: absolute;
  bottom: 20px; left: 10px;
  width: 70px; height: 70px;
  border-radius: 50%;
  background: #f1c40f;
}
```

Travail demandé

1. Créez trois animations : `traverser` (translateX(0) → translateX(500px)), `rougir` (background-color de #f1c40f vers #e74c3c) et `lueur` (box-shadow de 0 0 10px #f1c40f vers 0 0 40px #f39c12).
2. Appliquez les trois à .soleil : traverser et rougir pendant 6s avec `linear forwards`, lueur pendant 1s avec `ease-in-out infinite alternate`.
3. Observez : que fait le soleil pendant les 6 secondes ? Et après ?
4. Ajoutez une quatrième animation `grossir` (transform: scale(1) → scale(1.5), 6s, forwards). Que devient le déplacement ?
5. Corrigez le problème : placez le soleil dans un conteneur `.orbite` qui se déplace (traverser) ; gardez grossir, rougir et lueur sur .soleil.

Questions

Question 1 : Pourquoi le soleil ne se déplace-t-il plus après l'étape 4 alors que traverser est toujours dans la liste ? Quelle règle s'applique quand deux animations modifient la même propriété ?

Bonus

- Retardez la lueur d'une seconde : lueur 1s ease-in-out 1s infinite alternate. Chaque animation de la liste a sa propre durée, son propre délai et sa propre répétition.

Exercice 7 : Formes en mouvement : cercle, carré, triangle

Objectif : *déplacer des formes avec translate et contrôler leur répétition.*

Code de départ

Un cercle, un carré et un triangle réalisés en CSS :

```
<div class="scene">
  <div class="cercle"></div>
  <div class="carre2"></div>
  <div class="triangle"></div>
</div>
```

```
.scene {
  position: relative;
  width: 600px; height: 260px;
  border: 2px solid #333;
  background: #ecf0f1;
  overflow: hidden;
}
.cercle {
  position: absolute; top: 10px; left: 10px;
  width: 60px; height: 60px;
  border-radius: 50%;
  background: #3498db;
}
.carre2 {
  position: absolute; top: 100px; left: 10px;
  width: 60px; height: 60px;
  background: #e74c3c;
}
.triangle {
  position: absolute; top: 190px; left: 10px;
  width: 0; height: 0;
  border-left: 30px solid transparent;
  border-right: 30px solid transparent;
  border-bottom: 55px solid #f1c40f;
}
```

Travail demandé

1. **Cercle :** créez l'animation droite : il se déplace de gauche à droite (translateX(0) → translateX(520px)) en 3s, linear, et **reste à droite** à la fin.
2. **Carré :** créez l'animation aller-retour : il va à droite en tournant (translateX(520px) rotate(180deg)) puis revient, **sans jamais s'arrêter** (infinite et alternate), durée 2s.
3. **Triangle :** créez l'animation zigzag en 4 étapes : 0 % (0, 0) ; 33 % (170px, -60px) ; 66 % (350px, 0) ; 100 % (520px, -60px). Il doit **rester** à sa dernière position.

Indice : pour déplacer une forme, on utilise transform: translate(x, y). Pour plusieurs étapes, utilisez les pourcentages dans @keyframes.

Bonus

- Avec animation-delay, faites partir les trois formes l'une après l'autre (0s, 1s, 2s).

Exercice 8 : La roue qui avance et reste en rotation

Objectif : *faire avancer la roue jusqu'au bout de la route tout en la laissant tourner.*

Code de départ

```
<div class="route">
  <div class="support">
    <div class="roue"></div>
  </div>
</div>
```

```
.route {
  position: relative;
  width: 600px; height: 120px;
  border-bottom: 6px solid #555;
}
.support {
  position: absolute;
  bottom: 0; left: 0;
  width: 100px; height: 100px;
}
.roue {
  width: 100px; height: 100px;
  box-sizing: border-box;
  border: 6px solid #333;
  border-radius: 50%;
  background: conic-gradient(#e74c3c 0 25%, #f1c40f 0 50%,
                             #2ecc71 0 75%, #3498db 0);
}
```

Travail demandé

1. Créez `avancer` (translateX(0) → translateX(480px)) et appliquez-la à .support : 4s, linear, forwards.
2. Créez `tourner` (rotate(0deg) → rotate(360deg)) et appliquez-la à .roue : 1s, linear, infinite.
3. Observez : la roue s'arrête-t-elle de tourner quand elle arrive à droite ?

Questions

Question 1 : Pourquoi utilise-t-on deux éléments (.support et .roue) au lieu d'appliquer les deux animations sur la même div ?

Bonus

- La roue a un diamètre de 100px (circonférence ≈ 314px). Elle avance de 480px en 4s. Calculez la durée d'un tour pour qu'elle roule sans glisser.

Exercice 9 : Projet final : page d'accueil animée (flex, sticky, fixed)

Objectif : *réaliser une page d'accueil avec un en-tête qui reste visible, un menu et des cartes alignés avec flex, et un bouton « Haut de page » qui apparaît en douceur.*

Code de départ

Fichier index.html :

```
<body>
<header id="haut" class="entete">
  <h1 class="logo">Rafèha Info</h1>
  <nav class="menu">
    <a href="#accueil">Accueil</a>
    <a href="#cours">Cours</a>
    <a href="#contact">Contact</a>
  </nav>
</header>

<section id="accueil" class="hero">
  <h2 class="titre">Bienvenue au club d'informatique</h2>
  <p>Apprenez HTML5, CSS3 et JavaScript.</p>
</section>

<section id="cours">
  <h2>Nos cours</h2>
  <div class="cartes">
    <div class="carte2">HTML5</div>
    <div class="carte2">CSS3</div>
    <div class="carte2">JavaScript</div>
  </div>
</section>

<section id="contact" class="contact">
  <h2>Contact</h2>
  <p>Écrivez-nous pour vous inscrire.</p>
</section>

<a href="#haut" class="haut">↑ Haut</a>
</body>
```

Fichier style.css :

```
body { margin: 0; font-family: Arial, sans-serif; }
.entete { background: #2c3e50; padding: 15px 30px; }
.logo { margin: 0; color: #fff; font-size: 24px; }
.menu a { color: #fff; text-decoration: none; }
section { padding: 60px 30px; min-height: 350px; }
.hero { background: #ecf0f1; text-align: center; }
.carte2 {
  background: #3498db; color: #fff;
  padding: 40px 20px; width: 160px;
  text-align: center; border-radius: 8px;
}
.contact { background: #fdebd0; }
.haut {
  background: #e74c3c; color: #fff;
  padding: 10px 16px; border-radius: 6px;
  text-decoration: none;
}
```

Travail demandé

1. **flex :** dans .entete : `display: flex; justify-content: space-between; align-items: center;` (logo à gauche, menu à droite).
2. Dans .menu : `display: flex; gap: 20px;`.
3. Dans .cartes : `display: flex; gap: 20px; flex-wrap: wrap; justify-content: center;`.
4. **sticky :** dans .entete : `position: sticky; top: 0;`. Faites défiler la page.
5. Si le contenu passe par-dessus l'en-tête, ajoutez `z-index: 10;`.
6. **transitions :** liens du menu au survol : couleur `#f1c40f` et `transform: translateY(-3px)` (transition 0.3s).

7. Cartes au survol : `transform: scale(1.08)` et changement de fond (transition 0.3s).
8. **forwards :** créez `@keyframes apparition` (opacity 0 → 1, translateY(-40px) → translateY(0)) ; dans .titre : `opacity: 0;` puis `animation: apparition 1.5s ease forwards;`.
9. **fixed :** dans .haut : `position: fixed; bottom: 20px; right: 20px; opacity: 0;`.
10. Créez `@keyframes montrer` (opacity 0 → 1) et appliquez `animation: montrer 1s ease 2s forwards;` au bouton.
11. Faites défiler la page et cliquez sur le bouton « Haut de page ».

Questions

Question 1 : À quoi sert flex-wrap: wrap ? Réduisez la largeur de la fenêtre pour le constater.

Question 2 : Que se passe-t-il si on écrit position: sticky sans top: 0 ?

Question 3 : Le transform fonctionne sur les liens du menu parce qu'ils sont dans un conteneur flex. Un lien <a> seul (élément en ligne) ignorerait transform. Pourquoi flex règle-t-il ce problème ? (Indice : les enfants d'un conteneur flex deviennent des blocs.)

Question 4 : Quelle est la différence entre position: sticky et position: fixed ? Pourquoi forwards est-il indispensable sur le bouton ?

Bonus

- Ajoutez html { scroll-behavior: smooth; } pour un retour en haut fluide.
- Faites rebondir le bouton en continu : animation: montrer 1s ease 2s forwards, rebondir 1s ease-in-out 3s infinite alternate; (rebondir : translateY(0) → translateY(-10px)).

Enseignant : Aymen Essouyah